

TTB Label Verifier

Approach and design

Matthew Nolan · June 2026 · [live demo](#) · github.com/Mjnolan91/ttb-label-verifier

1. Summary

A compliance agent at TTB, the Alcohol and Tobacco Tax and Trade Bureau, uploads photos of an alcohol label. A vision model reads them into the full set of TTB-required fields, and deterministic code checks the label against the application and against TTB's rules, field by field: brand name, alcohol content within the legal tolerance for its class, the statutory government warning word for word, and the rest. The verdict is Approve, Needs review, or Reject, with a card per field saying how it was compared and why. A batch screen runs the same engine over a few hundred labels at once; every read exports as JSON or CSV.

You can check this yourself in two minutes. Open the live demo, click "Load the sample label", accept the AI's suggestions, and a real front-and-back spirits pair reads as one product and lands on Approve. Load the defective-warning version and the same flow ends in Reject, citing 27 CFR 16.22(a)(2), because that back label prints the warning prefix in title case. That is the exact rejection one of the interviewed agents described, reproduced live.

Three numbers carried the design. The suite is 811 tests that run offline in seconds with no API keys. CI fails the build if approve precision drops below 98 percent on the 27 labeled cases (it scores 100 today), because a false approval is the one error I decided this tool would not ship: an unnecessary review costs an agent minutes, a false approval costs the tool its trust. And the deployed demo measured a 3.1-second median (5.2 at p95) against the brief's five-second ceiling; the bundled clean pair runs six to nine seconds, and the defective pair closer to thirteen, because the tool takes one zoomed look at the warning before committing to a hard Reject.

2. What the customers said they wanted

Four voices run through the discovery notes, and each one set a requirement I treated as hard.

Sarah, the Deputy Director, said results must come back in about five seconds: the previous scanning vendor took 30 to 40 seconds per label and was abandoned. Her agents range from fresh graduates to a 73-year-old she considers the benchmark user, and importers dump 200 to 300 applications at once.

Marcus, in IT, explained that the outbound firewall had blocked the last vendor's ML endpoints, that they are an Azure shop, and that this prototype should stand alone, with no PII stored and no integration with COLA (TTB's label-approval filing system).

Dave, a 28-year agent, gave the false-rejection warning: STONE'S THROW and Stone's Throw are obviously the same product, and a tool doing rigid pattern matching would flood his queue with false rejections.

Jenny, a junior agent, described the strictest check: the government warning must match the statute word for word, with GOVERNMENT WARNING: in capital letters and bold; title case gets rejected. And badly photographed labels are out of scope but should fail gracefully.

So: a five-second budget, a screen a 73-year-old can use without hunting, batch at importer scale, extraction that survives the firewall, tolerant brand matching, a strict warning check, and graceful failure on bad photos. Jenny's last line became the error model for the whole tool. It is allowed to say it is not sure; it is never allowed to be silently wrong.

3. How I implemented it

AI extracts, code decides. The vision model has exactly one job: transcribe the label images into structured fields, each with a confidence. Every pass, review, or fail after that is plain, unit-tested TypeScript: comparators tolerant of case and punctuation (STONE'S THROW and Stone's Throw compare equal, and a near miss routes to review, not reject), a completeness matrix for what each beverage class must carry, and the statutory constants (the 27 CFR 16.21 warning text, the per-class alcohol tolerances) in one hand-verified module guarded by tests. A rejection therefore cites a rule, not a model's opinion.

Trust is measured, not assumed. Each product is read several times in parallel (front and back ride one request, so the model sees the panels together), and a field's confidence is the fraction of reads that agree. Anything still doubtful gets a bounded escalation: a stronger model re-reads exactly the doubtful fields, a dedicated judge decides whether the warning prefix is truly bold, and a missing warning gets one zoomed look at a cropped, straightened patch of the label. An escalation may clear a false alarm; none may flip a conflict to pass.

The screens stay out of the way. The single screen leads with the application comparison, per the brief. The AI's reading pre-fills every input as a gray suggestion accepted with Tab or one click, which removes the hand re-keying of label fields; one screen, nothing to hunt for, which was the 73-year-old test. A suggestion counts only once a person accepts it, so the model never approves its own work. Batch runs the same engine over a folder of images: fronts and backs pair by filename, and an optional CSV supplies claimed values. Rows stream into a reviewable worklist with automatic retries and a delayed second look at exactly the fields a clean read missed; a header toggle switches between the two screens.

Offline and deterministic. The default provider is a mock keyed off fixture filenames, so the 811 tests in 63 files and the 27-case eval run in seconds with zero keys, and CI runs the same gate (typecheck, lint, test, eval, build) on every push to main. Clone the repo, and npm test proves the whole pipeline on your laptop.

4. Why I chose this architecture

The AI models: a fast reader and an expensive judge. The demo runs OpenAI gpt-4.1 for extraction and gpt-5.5 only for the narrow judgments (the bold-prefix call, the focused re-reads). I measured the obvious alternative, running everything on the strongest model: identical verdicts, the median more than doubled (2.8 to 6.5 seconds), and the same experiment on Gemini failed 4 of its 9 requests on the Pro model's per-minute quota at that key's tier. Sarah's five-second ceiling made that decisive. Transcription is what current vision models are reliably good at, so the cheap model does the volume work and the expensive one is spent where a single judgment can hard-fail a label. The split held on both vendors I measured, and I trust sampling agreement over any model's self-reported confidence, so the trust mechanism itself is vendor-neutral.

A provider interface, because vendor lock-in already failed here once. The last vendor died at Marcus's firewall, so coupling to one endpoint was a stated failure mode, not a hypothetical. Extraction sits behind one VisionProvider interface with six modes: mock, OpenAI, Gemini,

Azure OpenAI, Azure Document Intelligence, and an ensemble that routes cross-provider disagreement to review. Production would point at the in-tenant Azure providers (the same model family, inside the tenant the firewall trusts); the public demo runs on a single OpenAI key; switching is an env-var change. The offline mock is the quiet workhorse of the design: it is what makes every test deterministic and lets a reviewer run the entire suite with no keys and no network.

Why Next.js. The honest alternatives were a React SPA with a separate API service, or a Python backend for the model calls. Both mean two deployables, two type systems, and API keys living in a second codebase. Next.js collapses that: one strict-TypeScript codebase where route handlers run the pipeline server-side (keys never reach the browser), server components render the screens, and the extracted-field shape is one set of types shared by the pipeline, the UI, and the CSV export, so the field list cannot drift between layers. The standalone build ships as a small container for the Azure target. It is also deliberately boring: a reviewer finds the API route and the page exactly where Next.js puts them. I wanted the plumbing unremarkable so the thinking worth reviewing could live in the domain logic.

The rest of the stack, briefly. TypeScript strict, because compliance code is exactly where an implicit any hides a bug. Tailwind, because the accessibility rules (4.5:1 contrast in both themes, 44-pixel targets, visible focus) are tokens applied at the point of use, where a stylesheet that drifts from its markup cannot hide. Vitest, because a suite that finishes in about five seconds gets run on every save, and a suite that gets run is the only kind that helps. sharp is the one extra runtime dependency, for the server-side crop and upscale behind the warning check. And no database, on purpose: Marcus said standalone with no PII, so nothing is stored and there is nothing to secure, back up, or migrate.

Why Vercel for the demo, Azure for production. I wanted a reviewer inside the running app one click after opening the repo, and Vercel auto-deploys this repo's main branch with zero configuration. It is only the demo host: the firewall requirement points production at in-tenant Azure, which is why the repo ships a multi-stage Dockerfile with documented App Service and Container Apps paths. Vercel is the shortest path to a working demo in your browser; Azure is where it would actually live.

Deterministic comparators, because the verdict must survive an audit. A compliance verdict has to be auditable and reproducible. "The model felt the brand matched" is not an answer a government reviewer can act on, so the model never makes the final call.

What I would do next. Deploy the container in-tenant with provisioned model quota behind the agency gateway (authentication and rate limiting), then layout-preserving OCR to unlock the type-size and placement rules that text-only extraction cannot check, then a sampled human-audit loop over auto-approvals so the false-approval rate keeps getting measured once real labels flow through it. COLA integration goes last; reaching that system is procurement work before it is engineering work.

One pattern runs through all of it: decide by measurement, gate the build on the error class you refuse to ship, and route uncertainty to people.